

Reviewer Pack — Minimal Order of Twisted Differential Forms and Circuit Mono...

Entry 066748a49d58 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 05:55:34 UTC

Minimal Order of Twisted Differential Forms and Circuit Monotone Width Correlation

Entry ID: 066748a49d58 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 05:55:34 UTC

1. Conjecture

Field A (mathematical branch): Twisted Differential Geometry **Field B** (complexity object): Boolean Circuits

Statement:

For a given Boolean circuit C with n inputs, the minimal order $m(C)$ of its associated twisted differential forms is linearly correlated with its circuit monotone width $w(C)$, such that $m(C) = \Theta(w(C))$. Furthermore, for any instance φ of size $n \leq 40$, if φ is satisfiable, then $m(C\varphi) < k * w(C\varphi)$, where k is a constant.

Rationale (proposer's reasoning):

Twisted differential geometry generalizes classical differential geometry by incorporating additional algebraic structures. The minimal order of twisted differential forms captures the complexity of these structures. Circuit monotone width measures the depth of the circuit, which directly relates to its computational difficulty. A strong correlation between these two invariants could reveal a deep connection between geometric properties and computational complexity.

Taxonomy category: Twisted Differential Geometry × Boolean Circuits (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e06f99f548688e7f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the instances with $n \leq 40$, when satisfiable, show $m(C\varphi) < k * w(C\varphi)$ for all seeds, and no seed produces a metric > 10 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "twisted differential geometry" AND "Boolean circuit monotone width"
- "minimal order twisted differential forms" AND "circuit monotone width" - "correlation
minimal order differential forms circuit monotone width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1,
elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        pivot = matrix[i][i]
        if pivot == 0:
            continue
        for j in range(i, n):
            matrix[i][j] /= pivot
        for j in range(n):
            if j != i and matrix[j][i] != 0:
                factor = matrix[j][i]
                for k in range(i, n):
                    matrix[j][k] -= factor * matrix[i][k]

def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
```

```

        for k in range(p):
            C[i][j] += A[i][k] * B[k][j]
    return C

def tseitin_formula(circuit):
    n = len(circuit)
    literals = [f"x{i+1}" for i in range(n)]
    clauses = []
    for i, gate in enumerate(circuit):
        if gate[0] == 'AND':
            a, b = gate[1], gate[2]
            literals[i] = f"y{i+1}"
            clauses.append([literals[a-1], literals[b-1], f"~{literals[i]}"])
        elif gate[0] == 'OR':
            a, b = gate[1], gate[2]
            literals[i] = f"y{i+1}"
            clauses.append([f"~{literals[a-1]}", literals[i]])
            clauses.append([f"~{literals[b-1]}", literals[i]])
        elif gate[0] == 'NOT':
            a = gate[1]
            literals[i] = f"y{i+1}"
            clauses.append([f"~{literals[a-1]}", literals[i]])
    return literals, clauses

def minimal_order(formula):
    n = len(formula)
    A = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        if formula[i].startswith('~'):
            j = int(formula[i][1:]) - 1
            A[j][i] = 1
        else:
            j = int(formula[i]) - 1
            A[j][j] += 1
    gaussian_elimination(A)
    rank = sum(1 for row in A if any(row))
    return rank

def circuit_monotone_width(circuit):
    n = len(circuit)
    max_depth = 0
    stack = [(0, 0)]
    while stack:
        node, depth = stack.pop()
        if node >= n:
            continue
        max_depth = max(max_depth, depth)
        for gate in circuit[node]:
            stack.append((gate-1, depth+1))
    return max_depth

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:

```

```

    for _ in range(5):
        circuit = [[random.choice(['AND', 'OR', 'NOT']), random.randint(1, n), random.randint(1, n)] for _ in range(n)]
        literals, clauses = tseitin_formula(circuit)
        formula = [l if not l.startswith('~') else f"~{l[1:]}" for l in literals]
        m_Cphi = minimal_order(formula)
        w_Cphi = circuit_monotone_width(circuit)
        results.append((m_Cphi, w_Cphi))
    mean_m = sum(m for m, _ in results) / len(results)
    mean_w = sum(w for _, w in results) / len(results)
    k = 1.5
    if all(m < k * w for m, w in results):
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "m(Cφ) >= k * w(Cφ)"
    return {
        "metric_name": "min_order_over_monotone_width",
        "metric_value": mean_m / mean_w,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean_metric = sum(results) / len(results)
    support_fraction = sum(1 for r in results if r < 10) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std=NA support_fraction={support_fraction}")
    elif any(r > 10 for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result > 10)
        print(f"RESULT: FALSIFIED counterexample=\"m(Cφ) > 10\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c5104e.py", line 129, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c5104e.py", line 104, in run_trial

```

```

m_Cphi = minimal_order(formula)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_03c5104e.py", line 76, in minimal_order
    j = int(formula[i]) - 1
    ^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'y1'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered support condition. | next: Investigate the cause of the crash and run the test again to gather the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14506
2	preregistration	ollama_remote	glm4:latest	0	0	8883
3	novelty	ollama_remote	glm4:latest	0	0	8042
4	novelty	ollama_remote	glm4:latest	0	0	10161
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21442
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19352
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21908
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30232
9	judge	ollama_remote	glm4:latest	0	0	9274

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143799 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/066748a49d58.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/066748a49d58.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/066748a49d58.tar.gz (if generated)